

Лабораторная работа (Вариант 3)

Python + PyQt6 + MySQL (VS Code)

Автор: Каламбет В.Б.

подготовка к демонстрационному экзамену

Содержание

Шаг 1. Создайте рабочую структуру проекта	4
Шаг 2. Поднимите MySQL + phpMyAdmin через Docker Compose	4
Шаг 3. Создайте схему БД (модули 1–4)	5
Шаг 4. Подготовьте CSV из исходных XLSX (Через Excel)	8
Шаг 5. Импортируйте CSV через UI phpMyAdmin	8
Шаг 5.1. Перенесите данные из raw в боевые таблицы	9
Шаг 6. Выполните контрольные SQL-проверки	11
Шаг 7. Создайте Python-проект и установите зависимости	12
Шаг 8. Создайте db.py	12
Шаг 8.1. Создайте check_db.py и проверьте подключение	13
Шаг 9. Создайте calculations.py	13
Шаг 10. Создайте main.py	16
Шаг 11. Запустите приложение	26
Шаг 12. Проверьте метод модуля 4	27
Шаг 13. Зафиксируйте метод модуля 4 в git	27
Шаг 14. Экспортируйте SQL-скрипт БД	28
Шаг 15. Сохраните ER-диаграмму БД в PDF	28
Шаг 16. Соберите исполняемый файл .exe	28
Шаг 17. Подготовьте финальный набор файлов для передачи	29

Результат лабораторной

После выполнения у вас будет готовое приложение по модулям 1–4:

- база данных MySQL в 3НФ;
 - ER-диаграмма БД в PDF;
 - импорт исходных данных из `xlsx` (через `csv`) в `phpMyAdmin`;
 - главная форма со списком заявок партнеров и расчетом стоимости заявки;
 - форма добавления/редактирования данных партнера;
 - окно продукции, входящей в заявку выбранного партнера;
 - метод расчета количества материала (модуль 4) и его локальный `git`-коммит;
 - SQL-скрипт БД, исполняемый файл приложения и набор итоговых файлов для передачи в предоставленный `git`-репозиторий (система контроля версий).
-

Шаг 1. Создайте рабочую структуру проекта

Что делаем

Создайте папку проекта и подкаталоги для кода и ресурсов.

Команды/код

```
cd C:\
mkdir demoexam_python_v3
cd demoexam_python_v3
mkdir resources
```

Скопируйте из папки Задание 3\Прил_В3_КОД 09.02.07-2-2025-ПУ\Ресурсы в resources:

- Новые технологии.png
 - Новые технологии.ico
 - все файлы *_import.xlsx
-

Шаг 2. Поднимите MySQL + phpMyAdmin через Docker Compose

Что делаем

Создайте docker-compose.yml и запустите контейнеры.

Альтернативно (без Docker) можно использовать XAMPP, Open Server Panel или локально установленный MySQL Server.

Команды/код

Создайте файл C:\demoexam_python_v3\docker-compose.yml:

```
services:
  mysql:
    image: mysql:8.4
    container_name: demoexam_v3_mysql
    restart: unless-stopped
    environment:
      MYSQL_ROOT_PASSWORD: root
      MYSQL_DATABASE: newtech_demo
      MYSQL_USER: demo
      MYSQL_PASSWORD: demo
    ports:
      - "3306:3306"
    volumes:
      - mysql_data_v3:/var/lib/mysql

  phpmyadmin:
    image: phpmyadmin:latest
    container_name: demoexam_v3_phpmyadmin
    restart: unless-stopped
```

```

environment:
  PMA_HOST: mysql
  PMA_PORT: 3306
  PMA_USER: root
  PMA_PASSWORD: root
ports:
  - "8081:80"
depends_on:
  - mysql

volumes:
  mysql_data_v3:

```

Запуск:

```

cd C:\demoexam_python_v3
docker compose up -d
docker compose ps

```

Шаг 3. Создайте схему БД (модули 1–4)

Что делаем

В phpMyAdmin создайте таблицы, связи, ограничения и служебные raw-таблицы для импорта.

Команды/код

1. Откройте phpMyAdmin:
 - при Docker: `http://localhost:8081`;
 - при XAMPP / Open Server Panel / локальном MySQL Server: адрес phpMyAdmin вашей локальной установки (часто `http://localhost/phpmyadmin`).
2. Войдите под пользователем с правами на создание таблиц (для Docker: root / root).
3. Если БД newtech_demo уже есть — выберите ее.
4. Если БД newtech_demo нет: вкладка **Базы данных** -> введите имя newtech_demo -> выберите сравнение `utf8mb4_unicode_ci` -> нажмите **Создать** -> откройте созданную БД.
5. Откройте вкладку **SQL** и выполните скрипт:

```
USE newtech_demo;
```

```
SET NAMES utf8mb4;
```

```

DROP TABLE IF EXISTS partner_products_requests;
DROP TABLE IF EXISTS products;
DROP TABLE IF EXISTS partners;
DROP TABLE IF EXISTS partner_types;
DROP TABLE IF EXISTS product_types;

```

```

DROP TABLE IF EXISTS material_types;
DROP TABLE IF EXISTS material_types_import_raw;
DROP TABLE IF EXISTS product_types_import_raw;
DROP TABLE IF EXISTS partners_import_raw;
DROP TABLE IF EXISTS products_import_raw;
DROP TABLE IF EXISTS partner_products_request_import_raw;

CREATE TABLE material_types (
    material_type_id INT AUTO_INCREMENT PRIMARY KEY,
    name VARCHAR(100) NOT NULL UNIQUE,
    defect_rate DECIMAL(10,6) NOT NULL CHECK (defect_rate >= 0)
) ENGINE=InnoDB DEFAULT CHARSET=utf8mb4;

CREATE TABLE product_types (
    product_type_id INT AUTO_INCREMENT PRIMARY KEY,
    name VARCHAR(100) NOT NULL UNIQUE,
    coefficient DECIMAL(10,4) NOT NULL CHECK (coefficient > 0)
) ENGINE=InnoDB DEFAULT CHARSET=utf8mb4;

CREATE TABLE partner_types (
    partner_type_id INT AUTO_INCREMENT PRIMARY KEY,
    name VARCHAR(50) NOT NULL UNIQUE
) ENGINE=InnoDB DEFAULT CHARSET=utf8mb4;

CREATE TABLE partners (
    partner_id INT AUTO_INCREMENT PRIMARY KEY,
    partner_type_id INT NOT NULL,
    name VARCHAR(255) NOT NULL UNIQUE,
    director VARCHAR(255) NOT NULL,
    email VARCHAR(255) NOT NULL,
    phone VARCHAR(50) NOT NULL,
    legal_address VARCHAR(500) NOT NULL,
    inn VARCHAR(12) NULL,
    rating INT NOT NULL CHECK (rating >= 0),
    CONSTRAINT fk_partners_type
        FOREIGN KEY (partner_type_id) REFERENCES partner_types(partner_type_id)
        ON UPDATE CASCADE ON DELETE RESTRICT
) ENGINE=InnoDB DEFAULT CHARSET=utf8mb4;

CREATE TABLE products (
    product_id INT AUTO_INCREMENT PRIMARY KEY,
    product_type_id INT NOT NULL,
    name VARCHAR(255) NOT NULL UNIQUE,
    article BIGINT NOT NULL UNIQUE CHECK (article > 0),
    min_partner_price DECIMAL(12,2) NOT NULL CHECK (min_partner_price >= 0),
    CONSTRAINT fk_products_type
        FOREIGN KEY (product_type_id) REFERENCES product_types(product_type_id)
        ON UPDATE CASCADE ON DELETE RESTRICT
) ENGINE=InnoDB DEFAULT CHARSET=utf8mb4;

```

```

CREATE TABLE partner_products_requests (
    partner_products_request_id INT AUTO_INCREMENT PRIMARY KEY,
    partner_id INT NOT NULL,
    product_id INT NOT NULL,
    quantity INT NOT NULL CHECK (quantity > 0),
    CONSTRAINT fk_ppr_partner
        FOREIGN KEY (partner_id) REFERENCES partners(partner_id)
        ON UPDATE CASCADE ON DELETE RESTRICT,
    CONSTRAINT fk_ppr_product
        FOREIGN KEY (product_id) REFERENCES products(product_id)
        ON UPDATE CASCADE ON DELETE RESTRICT,
    CONSTRAINT uk_ppr_partner_product UNIQUE (partner_id, product_id)
) ENGINE=InnoDB DEFAULT CHARSET=utf8mb4;

CREATE INDEX idx_partners_type ON partners(partner_type_id);
CREATE INDEX idx_products_type ON products(product_type_id);
CREATE INDEX idx_ppr_partner ON partner_products_requests(partner_id);
CREATE INDEX idx_ppr_product ON partner_products_requests(product_id);

-- гав-таблицы для импорта CSV в исходной форме
CREATE TABLE material_types_import_raw (
    material_type_name VARCHAR(255) NOT NULL,
    defect_percent_text VARCHAR(50) NOT NULL
) ENGINE=InnoDB DEFAULT CHARSET=utf8mb4;

CREATE TABLE product_types_import_raw (
    product_type_name VARCHAR(255) NOT NULL,
    coefficient_text VARCHAR(50) NOT NULL
) ENGINE=InnoDB DEFAULT CHARSET=utf8mb4;

CREATE TABLE partners_import_raw (
    partner_type_name VARCHAR(50) NOT NULL,
    partner_name VARCHAR(255) NOT NULL,
    director VARCHAR(255) NOT NULL,
    email VARCHAR(255) NOT NULL,
    phone VARCHAR(50) NOT NULL,
    legal_address VARCHAR(500) NOT NULL,
    inn VARCHAR(20) NOT NULL,
    rating_text VARCHAR(20) NOT NULL
) ENGINE=InnoDB DEFAULT CHARSET=utf8mb4;

CREATE TABLE products_import_raw (
    product_type_name VARCHAR(255) NOT NULL,
    product_name VARCHAR(255) NOT NULL,
    article_text VARCHAR(50) NOT NULL,
    min_partner_price_text VARCHAR(50) NOT NULL
) ENGINE=InnoDB DEFAULT CHARSET=utf8mb4;

```

```
CREATE TABLE partner_products_request_import_raw (  
    product_name VARCHAR(255) NOT NULL,  
    partner_name VARCHAR(255) NOT NULL,  
    quantity_text VARCHAR(50) NOT NULL  
) ENGINE=InnoDB DEFAULT CHARSET=utf8mb4;
```

Шаг 4 Подготовьте CSV из исходных XLSX (Через Excel)

Что делаем

Откройте каждый `xlsx` в Excel, сохраните как CSV UTF-8 и удалите строку заголовков.

Команды/код

Сохраните файлы:

- `Material_type_import.csv`
- `Product_type_import.csv`
- `Partners_import.csv`
- `Products_import.csv`
- `Partner_products_request_import.csv`

В каждом CSV удалите первую строку (заголовки), чтобы файл начинался сразу с данных.

Важно:

1. `Material_type_import.csv`:
 - Если Excel показывает 0,12%, сначала смените формат столбца на Общий или Числовой (значение должно стать долей, например 0,0012), затем замените запятую , на одну точку . (0.0012).
 - Допускается и формат 2E-3 (MySQL корректно обработает при переносе из raw).
 2. `Product_type_import.csv`:
 - Во втором столбце замените , на ..
 3. `Partners_import.csv`:
 - Проверьте, что ИНН не ушел в экспоненциальную запись (7.803E+09), а остался обычным числом.
 4. `Products_import.csv`:
 - В столбце цены замените , на ..
 5. `Partner_products_request_import.csv`:
 - Поле количества должно быть целым числом без дробной части.
-

Шаг 5. Импортируйте CSV через UI phpMyAdmin

Что делаем

Импортируйте CSV в raw-таблицы.

Команды/код

В phpMyAdmin:

1. Откройте таблицу `material_types_import_raw` -> **Импорт** -> файл `Material_type_import.csv` -> поле Названия столбцов: `material_type_name,defect_percent_text`.
2. Откройте таблицу `product_types_import_raw` -> **Импорт** -> файл `Product_type_import.csv` -> поле Названия столбцов: `product_type_name,coefficient_text`.
3. Откройте таблицу `partners_import_raw` -> **Импорт** -> файл `Partners_import.csv` -> поле Названия столбцов: `partner_type_name,partner_name,director,email,phone,legal_address,inn,ra`.
4. Откройте таблицу `products_import_raw` -> **Импорт** -> файл `Products_import.csv` -> поле Названия столбцов: `product_type_name,product_name,article_text,min_partner_price_text`.
5. Откройте таблицу `partner_products_request_import_raw` -> **Импорт** -> файл `Partner_products_request_import.csv` -> поле Названия столбцов: `product_name,partner_name,quantity_text`.

Для каждого импорта обязательно выставьте:

- Формат = CSV.
- Разделитель полей = , (для подготовленных CSV в этой ЛР). Если ваш CSV сохранен с ;, поставьте ;.
- Значения полей обрамлены = ".
- Символ экранирования = " (оставьте значение по умолчанию в вашей версии).
- Разделитель строк = auto.
- Названия столбцов — заполните вручную, как указано в пунктах 1–5 выше.

Шаг 5.1. Перенесите данные из raw в боевые таблицы

Что делаем

Заполните рабочие таблицы и связи.

Команды/код

Во вкладке **SQL** выполните:

```
USE newtech_demo;
```

```
-- 1) Минимальная нормализация названий в заявках (убираем двойные пробелы)
```

```
UPDATE partner_products_request_import_raw
SET product_name = REPLACE(product_name, ' ', '');
```

```
-- 2) Справочники
```

```
INSERT INTO partner_types (name)
SELECT DISTINCT TRIM(partner_type_name)
```

```

FROM partners_import_raw
WHERE TRIM(partner_type_name) <> '';

INSERT INTO material_types (name, defect_rate)
SELECT
    TRIM(material_type_name),
    CAST(REPLACE(REPLACE(TRIM(defect_percent_text), '%', ''), ', ', '.')) AS
DECIMAL(10,6))
FROM material_types_import_raw
WHERE TRIM(material_type_name) <> '';

INSERT INTO product_types (name, coefficient)
SELECT
    TRIM(product_type_name),
    CAST(REPLACE(TRIM(coefficient_text), ', ', '.')) AS DECIMAL(10,4))
FROM product_types_import_raw
WHERE TRIM(product_type_name) <> '';

-- 3) Основные таблицы
INSERT INTO partners (
    partner_type_id, name, director, email, phone, legal_address, inn, rating
)
SELECT
    pt.partner_type_id,
    TRIM(r.partner_name),
    TRIM(r.director),
    TRIM(r.email),
    TRIM(r.phone),
    TRIM(r.legal_address),
    NULLIF(TRIM(r.inn), ''),
    CAST(TRIM(r.rating_text) AS UNSIGNED)
FROM partners_import_raw r
JOIN partner_types pt ON pt.name = TRIM(r.partner_type_name)
WHERE TRIM(r.partner_name) <> '';

INSERT INTO products (
    product_type_id, name, article, min_partner_price
)
SELECT
    pt.product_type_id,
    TRIM(r.product_name),
    CAST(REPLACE(TRIM(r.article_text), ' ', '')) AS UNSIGNED),
    CAST(REPLACE(TRIM(r.min_partner_price_text), ', ', '.')) AS DECIMAL(12,2))
FROM products_import_raw r
JOIN product_types pt ON pt.name = TRIM(r.product_type_name)
WHERE TRIM(r.product_name) <> '';

INSERT INTO partner_products_requests (partner_id, product_id, quantity)
SELECT

```

```

        p.partner_id,
        pr.product_id,
        CAST(TRIM(r.quantity_text) AS UNSIGNED)
FROM partner_products_request_import_raw r
JOIN partners p ON p.name = TRIM(r.partner_name)
JOIN products pr ON pr.name = TRIM(r.product_name)
WHERE TRIM(r.quantity_text) <> ''
ON DUPLICATE KEY UPDATE quantity = VALUES(quantity);

```

Шаг 6. Выполните контрольные SQL-проверки

Что делаем

Проверьте, что импорт соответствует целевым количествам.

Команды/код

```
USE newtech_demo;
```

```

SELECT 'material_types' AS table_name, COUNT(*) AS cnt FROM material_types
UNION ALL
SELECT 'product_types', COUNT(*) FROM product_types
UNION ALL
SELECT 'partners', COUNT(*) FROM partners
UNION ALL
SELECT 'products', COUNT(*) FROM products
UNION ALL
SELECT 'partner_products_requests', COUNT(*) FROM partner_products_requests;

```

Контрольные количества: material_types=5, product_types=5, partners=20, products=20, partner_products_requests=80.

Дополнительная проверка расчетов:

```

SELECT
    p.name AS partner_name,
    ROUND(COALESCE(SUM(r.quantity * pr.min_partner_price), 0), 2) AS request_cost
FROM partners p
LEFT JOIN partner_products_requests r ON r.partner_id = p.partner_id
LEFT JOIN products pr ON pr.product_id = r.product_id
GROUP BY p.partner_id, p.name
ORDER BY request_cost DESC
LIMIT 5;

```

Что проверяет запрос:

- правильно ли связаны partners -> partner_products_requests -> products;
- правильно ли считается стоимость заявки как SUM(quantity * min_partner_price).

Если импорт и расчет верные, топ-5 будет таким:

- Декор и отделка -> 63193072.00

- Паркет -> 58785363.20
 - Гранит -> 58784800.00
 - Самоделка -> 57287880.00
 - СтройМастер -> 56481520.00
-

Шаг 7. Создайте Python-проект и установите зависимости

Что делаем

Создайте виртуальное окружение и установите пакеты.

Команды/код

```
cd C:\demoexam_python_v3
python -m venv .venv
.\venv\Scripts\activate
pip install PyQt6 mysql-connector-python pyinstaller
pip freeze > requirements.txt
```

Шаг 8. Создайте db.py

Что делаем

Вынесите подключение к БД в отдельный модуль.

Команды/код

Создайте C:\demoexam_python_v3\db.py:

```
import mysql.connector

DB_CONFIG = {
    "host": "127.0.0.1",
    "port": 3306,
    "user": "demo",
    "password": "demo",
    "database": "newtech_demo",
    "use_unicode": True,
    "charset": "utf8mb4",
    "use_pure": True,
}

def get_connection():
    return mysql.connector.connect(**DB_CONFIG)
```

Важно

db.py отдельно не запускайте: это модуль подключения.

Если используете ХАМПП: обычно user = "root", password = "".

Шаг 8.1. Создайте check_db.py и проверьте подключение

Что делаем

Создайте отдельный файл для проверки подключения к БД.

Команды/код

Создайте C:\demoexam_python_v3\check_db.py:

```
from db import get_connection

def main():
    conn = None
    cur = None
    try:
        conn = get_connection()
        cur = conn.cursor()
        cur.execute("SELECT 1")
        result = cur.fetchone()
        print(f"OK: подключение успешно, SELECT 1 -> {result[0]}")
    except Exception as e:
        print("ERROR:", e)
    finally:
        if cur is not None:
            cur.close()
        if conn is not None and conn.is_connected():
            conn.close()

if __name__ == "__main__":
    main()
```

Запуск:

```
cd C:\demoexam_python_v3
python check_db.py
```

Шаг 9. Создайте calculations.py

Что делаем

Реализуйте 2 расчетных метода:

- стоимость заявки партнера;
- количество необходимого материала (модуль 4).

Команды/код

Создайте C:\demoexam_python_v3\calculations.py:

```

from decimal import Decimal, ROUND_CEILING, ROUND_HALF_UP

from db import get_connection

def calculate_request_cost(items):
    """
    items: список пар (quantity, min_partner_price)
    Возвращает стоимость заявки с точностью до 0.01 и не ниже 0.
    """
    total = Decimal("0")

    for quantity, min_partner_price in items:
        qty = int(quantity)
        price = Decimal(str(min_partner_price))

        if qty < 0 or price < 0:
            continue

        total += Decimal(qty) * price

    if total < 0:
        total = Decimal("0")

    return total.quantize(Decimal("0.01"), rounding=ROUND_HALF_UP)

def calculate_required_material_count(
    product_type_id,
    material_type_id,
    required_products_count,
    products_in_stock,
    p1,
    p2,
):
    """
    Метод модуля 4.
    Возвращает -1 для невалидных входов/несуществующих типов.
    """
    try:
        product_type_id = int(product_type_id)
        material_type_id = int(material_type_id)
        required_products_count = int(required_products_count)
        products_in_stock = int(products_in_stock)
        p1 = Decimal(str(p1))
        p2 = Decimal(str(p2))
    except Exception:
        return -1

```

```

if (
    product_type_id <= 0
    or material_type_id <= 0
    or required_products_count < 0
    or products_in_stock < 0
    or p1 <= 0
    or p2 <= 0
):
    return -1

to_produce = max(required_products_count - products_in_stock, 0)
if to_produce == 0:
    return 0

conn = None
cur = None
try:
    conn = get_connection()
    cur = conn.cursor()

    cur.execute(
        "SELECT coefficient FROM product_types WHERE product_type_id = %s",
        (product_type_id,),
    )
    product_row = cur.fetchone()
    if not product_row:
        return -1
    coefficient = Decimal(str(product_row[0]))

    cur.execute(
        "SELECT defect_rate FROM material_types WHERE material_type_id = %s",
        (material_type_id,),
    )
    material_row = cur.fetchone()
    if not material_row:
        return -1
    defect_rate = Decimal(str(material_row[0]))

    # Формула модуля 4: учитываем количество к производству, коэффициент типа
    # и процент брака.
    required = (
        Decimal(to_produce)
        * p1
        * p2
        * coefficient
        * (Decimal("1") + defect_rate)
    )

    return int(required.to_integral_value(rounding=ROUND_CEILING))

```

```

except Exception:
    return -1
finally:
    if cur is not None:
        cur.close()
    if conn is not None and conn.is_connected():
        conn.close()

```

Шаг 10. Создайте main.py

Что делаем

Сделайте интерфейс: список заявок партнеров, форма добавления/редактирования, окно продукции выбранного партнера.

Команды/код

Создайте C:\demoexam_python_v3\main.py:

```

import os
import sys
from decimal import Decimal

from PyQt6.QtCore import Qt
from PyQt6.QtGui import QFont, QIcon, QPixmap
from PyQt6.QtWidgets import (
    QApplication,
    QComboBox,
    QDialog,
    QFormLayout,
    QFrame,
    QGridLayout,
    QHBoxLayout,
    QLabel,
    QLineEdit,
    QMainWindow,
    QMessageBox,
    QPushButton,
    QScrollArea,
    QSpinBox,
    QTableWidgetItem,
    QTableWidgetItem,
    QVBoxLayout,
    QWidget,
)

from calculations import calculate_request_cost
from db import get_connection

```



```

def resource_path(relative_path):
    if hasattr(sys, "_MEIPASS"):
        return os.path.join(sys._MEIPASS, relative_path)
    return os.path.join(os.path.abspath("."), relative_path)

class PartnerFormDialog(QDialog):
    def __init__(self, partner_id=None, parent=None):
        super().__init__(parent)
        self.partner_id = partner_id
        self.setWindowTitle("Добавление/редактирование заявки партнера")
        self.setMinimumWidth(700)
        self.setWindowIcon(QIcon(resource_path("resources/Новые
технологии.ico")))

        self.type_combo = QComboBox()
        self.name_edit = QLineEdit()
        self.director_edit = QLineEdit()
        self.address_edit = QLineEdit()
        self.rating_spin = QSpinBox()
        self.phone_edit = QLineEdit()
        self.email_edit = QLineEdit()

        self.rating_spin.setMinimum(0)
        self.rating_spin.setMaximum(1000)

        form = QFormLayout()
        form.setSpacing(10)
        form.addRow("Тип партнера:", self.type_combo)
        form.addRow("Наименование:", self.name_edit)
        form.addRow("ФИО директора:", self.director_edit)
        form.addRow("Юридический адрес:", self.address_edit)
        form.addRow("Рейтинг:", self.rating_spin)
        form.addRow("Телефон:", self.phone_edit)
        form.addRow("Email:", self.email_edit)

        save_btn = QPushButton("Сохранить")
        cancel_btn = QPushButton("Отмена")
        save_btn.clicked.connect(self.save_partner)
        cancel_btn.clicked.connect(self.reject)

        buttons = QHBoxLayout()
        buttons.addWidget(save_btn)
        buttons.addWidget(cancel_btn)

        root = QVBoxLayout(self)
        root.addLayout(form)
        root.addLayout(buttons)

```

```

        self.load_partner_types()
        if self.partner_id is not None:
            self.load_partner_data()

    def load_partner_types(self):
        conn = None
        cur = None
        try:
            conn = get_connection()
            cur = conn.cursor()
            cur.execute(
                "SELECT partner_type_id, name FROM partner_types ORDER BY
partner_type_id"
            )
            for type_id, name in cur.fetchall():
                self.type_combo.addItem(name, type_id)
        finally:
            if cur is not None:
                cur.close()
            if conn is not None and conn.is_connected():
                conn.close()

    def load_partner_data(self):
        conn = None
        cur = None
        try:
            conn = get_connection()
            cur = conn.cursor()
            cur.execute(
                """
                SELECT partner_type_id, name, director, legal_address, rating,
phone, email
                FROM partners
                WHERE partner_id = %s
                """,
                (self.partner_id,),
            )
            row = cur.fetchone()
            if not row:
                return

            partner_type_id, name, director, legal_address, rating, phone, email
= row

            idx = self.type_combo.findData(partner_type_id)
            if idx >= 0:
                self.type_combo.setCurrentIndex(idx)
            self.name_edit.setText(name)
            self.director_edit.setText(director)
            self.address_edit.setText(legal_address)

```

```

        self.rating_spin.setValue(int(rating))
        self.phone_edit.setText(phone)
        self.email_edit.setText(email)
    finally:
        if cur is not None:
            cur.close()
        if conn is not None and conn.is_connected():
            conn.close()

def save_partner(self):
    partner_type_id = self.type_combo.currentData()
    name = self.name_edit.text().strip()
    director = self.director_edit.text().strip()
    legal_address = self.address_edit.text().strip()
    rating = self.rating_spin.value()
    phone = self.phone_edit.text().strip()
    email = self.email_edit.text().strip()

    if not name:
        QMessageBox.warning(self, "Ошибка", "Введите наименование партнера.")
        return
    if not director:
        QMessageBox.warning(self, "Ошибка", "Введите ФИО директора.")
        return
    if not legal_address:
        QMessageBox.warning(self, "Ошибка", "Введите юридический адрес.")
        return
    if not phone:
        QMessageBox.warning(self, "Ошибка", "Введите телефон.")
        return
    if "@" not in email:
        QMessageBox.warning(self, "Ошибка", "Введите корректный email.")
        return

    conn = None
    cur = None
    try:
        conn = get_connection()
        cur = conn.cursor()

        if self.partner_id is None:
            cur.execute(
                """
                INSERT INTO partners (
                    partner_type_id, name, director, email, phone,
legal_address, inn, rating
                )
                VALUES (%s, %s, %s, %s, %s, %s, %s, %s)
                """
            )

```

```

        (partner_type_id, name, director, email, phone,
        legal_address, None, rating),
    )
    else:
        cur.execute(
            """
            UPDATE partners
            SET partner_type_id = %s,
              name = %s,
              director = %s,
              email = %s,
              phone = %s,
              legal_address = %s,
              rating = %s
            WHERE partner_id = %s
            """
            ,
            (partner_type_id, name, director, email, phone,
            legal_address, rating, self.partner_id),
        )

        conn.commit()
        self.accept()
    except Exception as e:
        QMessageBox.critical(self, "Ошибка", f"Не удалось сохранить данные:
        \n{e}")
    finally:
        if cur is not None:
            cur.close()
        if conn is not None and conn.is_connected():
            conn.close()

class PartnerProductsDialog(QDialog):
    def __init__(self, partner_id, partner_name, parent=None):
        super().__init__(parent)
        self.partner_id = partner_id
        self.setWindowTitle(f"Продукция в заявке: {partner_name}")
        self.setMinimumSize(850, 520)
        self.setWindowIcon(QIcon(resource_path("resources/Новые
        технологии.ico")))

        self.table = QTableWidget()
        self.table.setColumnCount(4)
        self.table.setHorizontalHeaderLabels(
            ["Наименование продукции", "Количество", "Мин. стоимость", "Сумма"]
        )
        self.table.horizontalHeader().setStretchLastSection(True)
        self.table.verticalHeader().setVisible(False)
        self.table.setEditTriggers(QTableWidget.EditTrigger.NoEditTriggers)

```

```

self.total_label = QLabel("Итого: 0.00 p")
self.total_label.setAlignment(Qt.AlignmentFlag.AlignRight)

close_btn = QPushButton("Закрыть")
close_btn.clicked.connect(self.accept)

root = QVBoxLayout(self)
root.addWidget(self.table)
root.addWidget(self.total_label)
root.addWidget(close_btn)

self.load_products()

def load_products(self):
    conn = None
    cur = None
    try:
        conn = get_connection()
        cur = conn.cursor()
        cur.execute(
            """
            SELECT pr.name, r.quantity, pr.min_partner_price
            FROM partner_products_requests r
            JOIN products pr ON pr.product_id = r.product_id
            WHERE r.partner_id = %s
            ORDER BY pr.name
            """
            ,
            (self.partner_id,),
        )
        rows = cur.fetchall()
    finally:
        if cur is not None:
            cur.close()
        if conn is not None and conn.is_connected():
            conn.close()

    self.table.setRowCount(len(rows))
    items_for_cost = []

    for i, (name, quantity, min_price) in enumerate(rows):
        line_sum = Decimal(str(quantity)) * Decimal(str(min_price))
        self.table.setItem(i, 0, QTableWidgetItem(str(name)))
        self.table.setItem(i, 1, QTableWidgetItem(str(quantity)))
        self.table.setItem(i, 2,
            QTableWidgetItem(f"{Decimal(str(min_price)):.2f}"))
        self.table.setItem(i, 3, QTableWidgetItem(f"{line_sum:.2f}"))
        items_for_cost.append((quantity, min_price))

```

```

        total = calculate_request_cost(items_for_cost)
        self.total_label.setText(f"Итого: {total:.2f} р")

class MainWindow(QMainWindow):
    def __init__(self):
        super().__init__()
        self.setWindowTitle("Заявки партнеров")
        self.setMinimumSize(1200, 780)
        self.setWindowIcon(QIcon(resource_path("resources/Новые
технологии.ico")))

        root = QWidget()
        root_layout = QVBoxLayout(root)
        root_layout.setSpacing(10)

        header = QHBoxLayout()
        logo = QLabel()
        pixmap = QPixmap(resource_path("resources/Новые технологии.png"))
        logo.setPixmap(pixmap.scaledToHeight(80,
Qt.TransformationMode.SmoothTransformation))
        logo.setStyleSheet("background: transparent;")

        add_btn = QPushButton("Добавить заявку")
        add_btn.clicked.connect(self.open_add_dialog)
        refresh_btn = QPushButton("Обновить")
        refresh_btn.clicked.connect(self.load_partners)

        header.addWidget(logo)
        header.addStretch()
        header.addWidget(add_btn)
        header.addWidget(refresh_btn)

        self.cards_layout = QVBoxLayout()
        self.cards_layout.setSpacing(12)
        self.cards_layout.setAlignment(Qt.AlignmentFlag.AlignTop)

        cards_widget = QWidget()
        cards_widget.setLayout(self.cards_layout)

        scroll = QScrollArea()
        scroll.setWidgetResizable(True)
        scroll.setWidget(cards_widget)
        scroll.setFrameShape(QFrame.Shape.NoFrame)

        root_layout.addLayout(header)
        root_layout.addWidget(scroll)
        self.setCentralWidget(root)

```

```

        self.load_partners()

def fetch_partners(self):
    conn = None
    cur = None
    try:
        conn = get_connection()
        cur = conn.cursor(dictionary=True)
        cur.execute(
            """
            SELECT
                p.partner_id,
                pt.name AS partner_type,
                p.name,
                p.director,
                p.email,
                p.phone,
                p.legal_address,
                p.rating
            FROM partners p
            JOIN partner_types pt ON pt.partner_type_id = p.partner_type_id
            ORDER BY p.partner_id
            """
        )
        return cur.fetchall()
    finally:
        if cur is not None:
            cur.close()
        if conn is not None and conn.is_connected():
            conn.close()

def fetch_partner_items(self, partner_id):
    conn = None
    cur = None
    try:
        conn = get_connection()
        cur = conn.cursor()
        cur.execute(
            """
            SELECT r.quantity, pr.min_partner_price
            FROM partner_products_requests r
            JOIN products pr ON pr.product_id = r.product_id
            WHERE r.partner_id = %s
            """
        ,
        (partner_id,))
    finally:
        return cur.fetchall()
    finally:
        if cur is not None:

```

```

        cur.close()
        if conn is not None and conn.is_connected():
            conn.close()

def clear_cards(self):
    while self.cards_layout.count():
        item = self.cards_layout.takeAt(0)
        w = item.widget()
        if w is not None:
            w.deleteLater()

def load_partners(self):
    self.clear_cards()
    for partner in self.fetch_partners():
        items = self.fetch_partner_items(partner["partner_id"])
        request_cost = calculate_request_cost(items)
        self.cards_layout.addWidget(self.build_card(partner, request_cost))

def build_card(self, partner, request_cost):
    card = QFrame()
    card.setObjectName("card")
    card.setFrameShape(QFrame.Shape.StyledPanel)

    layout = QGridLayout(card)
    layout.setColumnStretch(0, 4)
    layout.setColumnStretch(1, 2)

    left = QVBoxLayout()
    title = QLabel(f'{partner["partner_type"]} | {partner["name"]}')
    title.setObjectName("cardTitle")
    left.addWidget(title)
    left.addWidget(QLabel(f'ФИО директора: {partner["director"]}'))
    left.addWidget(QLabel(f'Телефон: {partner["phone"]}'))
    left.addWidget(QLabel(f'Email: {partner["email"]}'))
    left.addWidget(QLabel(f'Рейтинг: {partner["rating"]}'))

    right = QVBoxLayout()
    cost = QLabel(f"Стоимость заявки: {request_cost:.2f} p")
    cost.setObjectName("costLabel")
    cost.setAlignment(Qt.AlignmentFlag.AlignRight)

    edit_btn = QPushButton("Редактировать")
    edit_btn.clicked.connect(
        lambda _, pid=partner["partner_id"]: self.open_edit_dialog(pid)
    )
    products_btn = QPushButton("Продукция")
    products_btn.clicked.connect(
        lambda _, pid=partner["partner_id"], name=partner["name"]:
self.open_products_dialog(pid, name)

```



```

    )

    right.addWidget(cost)
    right.addWidget(edit_btn)
    right.addWidget(products_btn)
    right.addStretch()

    layout.addLayout(left, 0, 0)
    layout.addLayout(right, 0, 1)
    return card

def open_add_dialog(self):
    dlg = PartnerFormDialog(parent=self)
    if dlg.exec():
        self.load_partners()

def open_edit_dialog(self, partner_id):
    dlg = PartnerFormDialog(partner_id=partner_id, parent=self)
    if dlg.exec():
        self.load_partners()

def open_products_dialog(self, partner_id, partner_name):
    dlg = PartnerProductsDialog(partner_id, partner_name, parent=self)
    dlg.exec()

def apply_style(app):
    app.setFont(QFont("Bahnschrift Light SemiCondensed", 12))
    app.setStyleSheet(
        """
        QWidget {
            background: #FFFFFF;
            color: #1a1a1a;
        }
        QFrame#card {
            background: #BBDCEA;
            border: 1px solid #0C4882;
            border-radius: 8px;
            padding: 10px;
        }
        QLabel#cardTitle {
            font-size: 22px;
            font-weight: 700;
        }
        QLabel#costLabel {
            font-size: 20px;
            font-weight: 700;
            color: #0C4882;
        }
        """
    )

```

```

        QPushButton {
            background: #0C4882;
            color: white;
            border: none;
            border-radius: 6px;
            padding: 8px 14px;
            min-height: 34px;
        }
        QPushButton:hover {
            background: #0a3f70;
        }
        QLineEdit, QComboBox, QSpinBox, QTableWidget {
            border: 1px solid #0C4882;
            border-radius: 5px;
            padding: 6px;
            background: white;
        }
    """
)

def main():
    app = QApplication(sys.argv)
    apply_style(app)
    window = MainWindow()
    window.show()
    sys.exit(app.exec())

if __name__ == "__main__":
    main()

```

Шаг 11. Запустите приложение

Что делаем

Проверьте, что форма открывается и список партнеров загружается из БД. Проверьте возврат на главную форму через кнопки Отмена и Закрывать в дочерних окнах.

Команды/код

```

cd C:\demoexam_python_v3
.venv\Scripts\activate
python main.py

```

Шаг 12. Проверьте метод модуля 4

Что делаем

Проверьте `calculate_required_material_count(...)` на валидных и невалидных данных.

Команды/код

Создайте `C:\demoexam_python_v3\check_module4.py`:

```
from calculations import calculate_required_material_count

print(calculate_required_material_count(1, 1, 1000, 200, 2.5, 1.5)) # ожидаем
4509
print(calculate_required_material_count(2, 4, 5000, 1200, 1.2, 2.1)) # ожидаем
33567
print(calculate_required_material_count(4, 5, 12000, 500, 3.0, 0.8)) # ожидаем
124424
print(calculate_required_material_count(5, 3, 2500, 2500, 2.5, 2.5)) # ожидаем
0
print(calculate_required_material_count(99, 1, 100, 0, 1.0, 1.0)) # ожидаем
-1
print(calculate_required_material_count(1, 1, 100, 0, -1.0, 2.0)) # ожидаем
-1
```

Запуск:

```
cd C:\demoexam_python_v3
python check_module4.py
```

Шаг 13. Зафиксируйте метод модуля 4 в git

Что делаем

Зафиксируйте коммит после реализации метода модуля 4. По условию Варианта 3 исходник метода нужно передать отдельным репозиторием с именем проекта.

Команды/код

```
cd C:\demoexam_python_v3
git add calculations.py check_module4.py
git commit -m "Добавлен метод расчета количества материала (модуль 4)"
```

В отдельном репозитории метода (имя репозитория = имя проекта) зафиксируйте `calculations.py`:

```
git add calculations.py
git commit -m "Метод модуля 4"
git push
```

Шаг 14. Экспортируйте SQL-скрипт БД

Что делаем

Сохраните итоговый SQL-скрипт из phpMyAdmin.

Команды/код

1. В phpMyAdmin выберите БД newtech_demo.
 2. Откройте вкладку **Экспорт**.
 3. В блоке Метод экспорта выберите Обычный - отображать все возможные настройки.
 4. Проверьте:
 - Формат = SQL;
 - в блоке Таблицы включены Структура и Данные;
 - в блоке Параметры создания объектов при необходимости включены Добавить выражение DROP TABLE / VIEW / PROCEDURE / FUNCTION / EVENT / TRIGGER и IF NOT EXISTS;
 - в блоке Параметры создания данных оператор = INSERT.
 5. В блоке Вывод оставьте Сохранить вывод в файл.
 6. Нажмите **Экспорт** и сохраните файл как newtech_demo.sql.
-

Шаг 15. Сохраните ER-диаграмму БД в PDF

Что делаем

Получите ER-диаграмму средствами phpMyAdmin и сохраните в PDF.

Команды/код

1. В phpMyAdmin откройте БД newtech_demo.
 2. В верхнем меню БД выберите **Ещё** -> **Дизайнер**.
 3. В дизайнере нажмите **Экспорт схемы**.
 4. В поле формата выберите PDF.
 5. Сохраните файл как newtech_demo_er.pdf.
-

Шаг 16. Соберите исполняемый файл .exe

Что делаем

Сформируйте исполняемый файл приложения Python.

Команды/код

```
cd C:\demoexam_python_v3
pyinstaller --noconfirm --windowed --onefile --name NewTechPartnersApp --icon
"resources\Новые технологии.ico" --add-data "resources;resources" --collect-all
mysql.connector main.py
```

Важно

Перед повторной сборкой закройте запущенный NewTechPartnersApp.exe, иначе возможна ошибка WinError 5 (Отказано в доступе).

Шаг 17. Подготовьте финальный набор файлов для передачи

Что делаем

Соберите итоговые файлы точно в формате задания.

Команды/код

Подготовьте в папке проекта C:\demoexam_python_v3:

- папка исходного кода C:\demoexam_python_v3 (структура файлов, не архив);
- C:\demoexam_python_v3\dist\NewTechPartnersApp.exe;
- newtech_demo.sql (скопируйте экспортированный файл в C:\demoexam_python_v3);
- newtech_demo_er.pdf (скопируйте экспортированный файл в C:\demoexam_python_v3);
- прочие графические/текстовые файлы по условию площадки (если они требуются отдельно).

Зафиксируйте итог в git и подготовьте набор к передаче в предоставленный git-репозиторий (система контроля версий):

```
cd C:\demoexam_python_v3
git add .
git status
git commit -m "Финальная версия проекта (Python, Вариант 3)"
git push
```